

图论 C++ 知识点总结

数据结构考试复习

2026 年 6 月 18 日

目录

1	图的存储	3
1.1	邻接矩阵 (Adjacency Matrix)	3
1.2	邻接表 (Adjacency List)	3
1.3	结构体封装图类	4
1.4	边集数组 (用于 Kruskal 算法)	5
2	并查集 (Union-Find / Disjoint Set Union)	6
2.1	标准实现 (路径压缩 + 按秩合并)	6
2.2	简化实现 (仅路径压缩)	7
2.3	应用: 统计连通分量	7
2.4	应用: 分组处理	8
3	图的遍历	9
3.1	深度优先搜索 (DFS)	9
3.2	广度优先搜索 (BFS)	9
3.3	BFS 求最短路径 (无权图)	10
3.4	迷宫 BFS (二维网格)	11
3.5	三维 BFS	12
4	拓扑排序 (Topological Sort)	13
4.1	Kahn 算法 (BFS 队列实现)	13
4.2	栈实现	13
5	最短路径	15
5.1	Dijkstra 算法 (单源最短路, 非负权)	15
5.1.1	朴素版 $O(n^2)$	15
5.1.2	堆优化版 $O(m \log n)$	15
5.2	Floyd-Warshall 算法 (多源最短路)	17
5.3	Bellman-Ford 算法 (含负权)	17

6 最小生成树 (Minimum Spanning Tree)	18
6.1 Kruskal 算法 (加边法, $O(m \log m)$)	18
6.2 Prim 算法 (加点法, $O(n^2)$)	18
7 关键路径 (AOE 网)	20
8 常见技巧与模板	21
8.1 快速 I/O	21
8.2 INF 常量	21
8.3 重边处理	21
8.4 邻接表排序	21
8.5 DFS 检测有向环	21
8.6 索引排序	22
8.7 优先队列 (堆)	22
附录: 算法速查表	23

1 图的存储

1.1 邻接矩阵 (Adjacency Matrix)

适用于稠密图，空间复杂度 $O(n^2)$ 。

Listing 1: 邻接矩阵存储

```
1  const int N = 1005;
2  const int INF = 0x3f3f3f3f;
3
4  int graph[N][N]; // graph[u][v] = 边权
5
6  // 初始化
7  void initGraph(int n) {
8      for (int i = 1; i <= n; ++i)
9          for (int j = 1; j <= n; ++j)
10             graph[i][j] = (i == j) ? 0 : INF;
11 }
12
13 // 添加有向边
14 void addEdge(int u, int v, int w) {
15     graph[u][v] = w;
16 }
17
18 // 添加无向边
19 void addUndirectedEdge(int u, int v, int w) {
20     graph[u][v] = w;
21     graph[v][u] = w;
22 }
```

1.2 邻接表 (Adjacency List)

适用于稀疏图，空间复杂度 $O(n + m)$ 。C++ 常用 `vector` 实现。

Listing 2: 邻接表存储 (vector 实现)

```
1  #include <vector>
2  using namespace std;
3
4  // 无权图：用 vector<vector<int>>
5  vector<vector<int>> adj;
6
7  void initGraph(int n) {
```

```

8      adj.resize(n);
9  }
10
11 void addEdge(int u, int v) {
12     adj[u].push_back(v);
13     adj[v].push_back(u); // 无向图
14 }
15
16 // 有权图
17 struct Edge {
18     int to, w;
19 };
20 vector<vector<Edge>> adj2;
21
22 void initWeightedGraph(int n) {
23     adj2.resize(n);
24 }
25
26 void addWeightedEdge(int u, int v, int w) {
27     adj2[u].push_back({v, w});
28     adj2[v].push_back({u, w}); // 无向图
29 }
30
31 // 遍历邻接点
32 for (int v : adj[u]) { /* ... */ }
33 for (Edge& e : adj2[u]) { /* e.to, e.w */ }

```

1.3 结构体封装图类

Listing 3: 图类封装

```

1 struct ALGraph {
2     vector<int> vertices[20];
3     int vernum, arcnum;
4 };
5
6 void CreateGraph(ALGraph& G, int n, int e) {
7     G.vernum = n;
8     G.arcnum = e;
9     for (int i = 0; i < e; i++) {
10         int u, v;
11         cin >> u >> v;

```

```

12         G.vertices[u].push_back(v);
13         G.vertices[v].push_back(u);    // 无向图
14     }
15 }

```

1.4 边集数组（用于 Kruskal 算法）

Listing 4: 边集存储

```

1  struct Edge {
2      int u, v, w;
3      bool operator<(const Edge& other) const {
4          return w < other.w;    // 按权值排序
5      }
6  };
7
8  vector<Edge> edges;
9  edges.push_back({u, v, w});
10 sort(edges.begin(), edges.end());

```

2 并查集 (Union-Find / Disjoint Set Union)

用于管理元素分组，支持查找和合并操作，近似 $O(\alpha(n))$ 。

2.1 标准实现 (路径压缩 + 按秩合并)

Listing 5: 并查集结构体

```
1 struct UnionFind {
2     vector<int> parent; // 父节点
3     vector<int> rank;   // 秩 (树高)
4
5     UnionFind(int n) {
6         parent.resize(n);
7         rank.resize(n, 0);
8         for (int i = 0; i < n; i++)
9             parent[i] = i;
10    }
11
12    // 查找根 + 路径压缩
13    int find(int x) {
14        if (parent[x] != x)
15            parent[x] = find(parent[x]);
16        return parent[x];
17    }
18
19    // 按秩合并
20    void unite(int x, int y) {
21        int rx = find(x), ry = find(y);
22        if (rx == ry) return;
23        if (rank[rx] < rank[ry])
24            parent[rx] = ry;
25        else if (rank[rx] > rank[ry])
26            parent[ry] = rx;
27        else {
28            parent[ry] = rx;
29            rank[rx]++;
30        }
31    }
32
33    bool same(int x, int y) {
34        return find(x) == find(y);
35    }
36}
```

```
35     }
36 };
```

2.2 简化实现（仅路径压缩）

Listing 6: 简单并查集

```
1  vector<int> parent;
2
3  void init(int n) {
4      parent.resize(n + 1);
5      for (int i = 1; i <= n; i++)
6          parent[i] = i;
7  }
8
9  int find(int x) {
10     if (parent[x] != x)
11         parent[x] = find(parent[x]);
12     return parent[x];
13 }
14
15 void unite(int x, int y) {
16     int rx = find(x), ry = find(y);
17     if (rx != ry) parent[ry] = rx;
18 }
19
20 bool same(int x, int y) {
21     return find(x) == find(y);
22 }
```

2.3 应用：统计连通分量

Listing 7: 并查集统计连通分量

```
1  // 不同的根节点数 = 连通分量数
2  int countComponents(int n) {
3      init(n);
4      for (每条边(u, v)) unite(u, v);
5      int cnt = 0;
6      for (int i = 1; i <= n; i++)
7          if (find(i) == i) cnt++;
8      return cnt;
```

```
9 }
```

2.4 应用：分组处理

Listing 8: 并查集分组（如家族问题）

```
1 map<int, vector<int>> groups;
2 for (int i = 0; i < n; i++) {
3     int root = uf.find(i);
4     groups[root].push_back(i);
5 }
6 for (auto& entry : groups) {
7     // entry.first = 根节点
8     // entry.second = 该组所有成员
9 }
```


3 图的遍历

3.1 深度优先搜索（DFS）

Listing 9: DFS 递归 / 非递归

```
1  #include <vector>
2  #include <stack>
3  using namespace std;
4
5  // 递归
6  void dfs(int u, vector<vector<int>>& adj, vector<bool>& visited) {
7      visited[u] = true;
8      cout << u << " ";
9      for (int v : adj[u])
10         if (!visited[v]) dfs(v, adj, visited);
11 }
12
13 // 非递归（栈）
14 void dfsStack(int start, vector<vector<int>>& adj) {
15     int n = adj.size();
16     vector<bool> vis(n, false);
17     stack<int> st;
18     st.push(start);
19     while (!st.empty()) {
20         int u = st.top(); st.pop();
21         if (vis[u]) continue;
22         vis[u] = true;
23         cout << u << " ";
24         for (int i = adj[u].size() - 1; i >= 0; i--) {
25             int v = adj[u][i];
26             if (!vis[v]) st.push(v);
27         }
28     }
29 }
```

3.2 广度优先搜索（BFS）

Listing 10: BFS 实现（含非连通图处理）

```
1  #include <queue>
2  #include <vector>
```

```

3  #include <iostream>
4  using namespace std;
5
6  void bfs(int start, vector<vector<int>>& adj,
7          vector<bool>& visited, vector<int>& traversal) {
8      queue<int> q;
9      q.push(start);
10     visited[start] = true;
11     while (!q.empty()) {
12         int u = q.front(); q.pop();
13         traversal.push_back(u);
14         for (int v : adj[u]) {
15             if (!visited[v]) {
16                 visited[v] = true;
17                 q.push(v);
18             }
19         }
20     }
21 }
22
23 // 非连通图：每个未访问点为起点做 BFS
24 void graphBFS(int n, vector<vector<int>>& adj) {
25     vector<bool> visited(n, false);
26     vector<int> traversal;
27     int components = 0;
28     for (int i = 0; i < n; i++) {
29         if (!visited[i]) {
30             bfs(i, adj, visited, traversal);
31             components++;
32         }
33     }
34     // traversal: BFS 序列
35     // components: 连通分量数
36 }

```

3.3 BFS 求最短路径（无权图）

Listing 11: BFS 求最短路径步数

```

1  vector<int> bfsShortestPath(int start, int n, vector<vector<int>>& adj) {
2      vector<int> dist(n, -1); // -1 表示不可达
3      queue<int> q;

```

```

4     dist[start] = 0;
5     q.push(start);
6     while (!q.empty()) {
7         int u = q.front(); q.pop();
8         for (int v : adj[u]) {
9             if (dist[v] == -1) {
10                 dist[v] = dist[u] + 1;
11                 q.push(v);
12             }
13         }
14     }
15     return dist;
16 }

```

3.4 迷宫 BFS（二维网格）

Listing 12: 二维迷宫 / 网格 BFS

```

1 struct Point { int x, y, step; };
2
3 int bfsMaze(vector<vector<int>>& maze, int sx, int sy,
4             int ex, int ey) {
5     int n = maze.size(), m = maze[0].size();
6     int dx[4] = {-1, 1, 0, 0};
7     int dy[4] = {0, 0, -1, 1};
8
9     queue<Point> q;
10    vector<vector<bool>> vis(n, vector<bool>(m, false));
11    q.push({sx, sy, 0});
12    vis[sx][sy] = true;
13
14    while (!q.empty()) {
15        Point cur = q.front(); q.pop();
16        if (cur.x == ex && cur.y == ey) return cur.step;
17
18        for (int i = 0; i < 4; i++) {
19            int nx = cur.x + dx[i], ny = cur.y + dy[i];
20            if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
21            if (maze[nx][ny] == 1 || vis[nx][ny]) continue;
22            vis[nx][ny] = true;
23            q.push({nx, ny, cur.step + 1});
24        }

```

```

25     }
26     return -1; // 不可达
27 }

```

3.5 三维 BFS

Listing 13: 三维 BFS (6 方向)

```

1  struct Point3D { int x, y, z; };
2  int dx[6] = {1, -1, 0, 0, 0, 0};
3  int dy[6] = {0, 0, 1, -1, 0, 0};
4  int dz[6] = {0, 0, 0, 0, 1, -1};
5
6  int bfs3D(vector<vector<vector<int>>>& mat,
7           vector<vector<vector<bool>>>& vis,
8           int M, int N, int L, int sx, int sy, int sz) {
9      queue<Point3D> q;
10     q.push({sx, sy, sz});
11     vis[sx][sy][sz] = true;
12     int vol = 1;
13     while (!q.empty()) {
14         Point3D p = q.front(); q.pop();
15         for (int i = 0; i < 6; i++) {
16             int nx = p.x + dx[i], ny = p.y + dy[i], nz = p.z + dz[i];
17             if (nx < 0 || nx >= M || ny < 0 || ny >= N || nz < 0 || nz >=
                L)
18                 continue;
19             if (mat[nx][ny][nz] == 1 && !vis[nx][ny][nz]) {
20                 vis[nx][ny][nz] = true;
21                 q.push({nx, ny, nz});
22                 vol++;
23             }
24         }
25     }
26     return vol;
27 }

```

4 拓扑排序 (Topological Sort)

有向无环图 (DAG) 的顶点线性排序, 每条有向边的起点都在终点之前。

4.1 Kahn 算法 (BFS 队列实现)

Listing 14: 拓扑排序 (Kahn 算法)

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  vector<int> topologicalSort(int n, vector<vector<int>>& adj) {
7      vector<int> indeg(n + 1, 0);
8      for (int u = 1; u <= n; u++)
9          for (int v : adj[u]) indeg[v]++;
10
11     queue<int> q;
12     for (int i = 1; i <= n; i++)
13         if (indeg[i] == 0) q.push(i);
14
15     vector<int> result;
16     while (!q.empty()) {
17         int u = q.front(); q.pop();
18         result.push_back(u);
19         for (int v : adj[u]) {
20             indeg[v]--;
21             if (indeg[v] == 0) q.push(v);
22         }
23     }
24
25     // 结果长度 != n 表示有环
26     return result.size() == n ? result : vector<int>{};
27 }
```

4.2 栈实现

Listing 15: 拓扑排序 (栈实现)

```
1  vector<int> topoSortByStack(int n, vector<vector<int>>& adj) {
2      vector<int> indeg(n + 1, 0);
```

```

3     for (int u = 1; u <= n; u++)
4         for (int v : adj[u]) indeg[v]++;
5
6     stack<int> st;
7     for (int i = 1; i <= n; i++)
8         if (indeg[i] == 0) st.push(i);
9
10    int count = 0;
11    vector<int> result;
12    while (!st.empty()) {
13        int u = st.top(); st.pop();
14        result.push_back(u);
15        count++;
16        for (int v : adj[u]) {
17            indeg[v]--;
18            if (indeg[v] == 0) st.push(v);
19        }
20    }
21    if (count != n) return {}; // 有环
22    return result;
23 }

```

5 最短路径

5.1 Dijkstra 算法（单源最短路，非负权）

5.1.1 朴素版 $O(n^2)$

Listing 16: Dijkstra 朴素版

```
1  #include <iostream>
2  #include <vector>
3  #include <climits>
4  using namespace std;
5
6  const int INF = INT_MAX / 2;
7
8  vector<int> dijkstra(int n, int start, vector<vector<int>>& graph) {
9      vector<int> dist(n + 1, INF);
10     vector<bool> visited(n + 1, false);
11     dist[start] = 0;
12
13     for (int i = 1; i <= n; i++) {
14         int u = -1, minDist = INF;
15         for (int j = 1; j <= n; j++)
16             if (!visited[j] && dist[j] < minDist) {
17                 minDist = dist[j];
18                 u = j;
19             }
20         if (u == -1) break;
21         visited[u] = true;
22
23         for (int v = 1; v <= n; v++)
24             if (!visited[v] && graph[u][v] != INF
25                 && dist[u] + graph[u][v] < dist[v])
26                 dist[v] = dist[u] + graph[u][v];
27     }
28     return dist;
29 }
```

5.1.2 堆优化版 $O(m \log n)$

Listing 17: Dijkstra 堆优化

```
1  #include <queue>
```

```

2  #include <vector>
3  using namespace std;
4
5  const int INF = 0x3f3f3f3f;
6  struct Edge { int to, w; };
7  struct Node {
8      int id, dist;
9      bool operator>(const Node& o) const {
10         return dist > o.dist;
11     }
12 };
13
14 vector<vector<Edge>> adj;
15
16 vector<int> dijkstraHeap(int n, int start) {
17     adj.resize(n + 1);
18     vector<int> dist(n + 1, INF);
19     vector<bool> vis(n + 1, false);
20     priority_queue<Node, vector<Node>, greater<Node>> pq;
21
22     dist[start] = 0;
23     pq.push({start, 0});
24
25     while (!pq.empty()) {
26         Node cur = pq.top(); pq.pop();
27         int u = cur.id;
28         if (vis[u]) continue;
29         vis[u] = true;
30         for (Edge& e : adj[u]) {
31             int v = e.to, w = e.w;
32             if (dist[u] + w < dist[v]) {
33                 dist[v] = dist[u] + w;
34                 pq.push({v, dist[v]});
35             }
36         }
37     }
38     return dist;
39 }

```


5.2 Floyd-Warshall 算法（多源最短路）

$O(n^3)$, 适用于 $n \leq 500$ 。

Listing 18: Floyd 算法

```
1  const int INF = 0x3f3f3f3f;
2  int dist[505][505];
3
4  void floyd(int n) {
5      for (int k = 1; k <= n; k++)
6          for (int i = 1; i <= n; i++)
7              for (int j = 1; j <= n; j++)
8                  if (dist[i][k] != INF && dist[k][j] != INF)
9                      dist[i][j] = min(dist[i][j],
10                                         dist[i][k] + dist[k][j]);
11 }
```

5.3 Bellman-Ford 算法（含负权）

Listing 19: Bellman-Ford 算法

```
1  struct Edge { int u, v, w; };
2  vector<Edge> edges;
3
4  bool bellmanFord(int n, int start, vector<int>& dist) {
5      dist.assign(n + 1, INF);
6      dist[start] = 0;
7      for (int i = 1; i < n; i++) {
8          bool updated = false;
9          for (Edge& e : edges)
10             if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
11                 dist[e.v] = dist[e.u] + e.w;
12                 updated = true;
13             }
14             if (!updated) break;
15     }
16     // 第 n 轮检测负环
17     for (Edge& e : edges)
18         if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v])
19             return false; // 有负环
20     return true;
21 }
```

6 最小生成树 (Minimum Spanning Tree)

6.1 Kruskal 算法 (加边法, $O(m \log m)$)

基于并查集, 按边权从小到大选取。

Listing 20: Kruskal 算法

```
1  #include <algorithm>
2  #include <vector>
3  using namespace std;
4
5  struct Edge {
6      int u, v, w;
7      bool operator<(const Edge& o) const { return w < o.w; }
8  };
9
10 // 并查集 (见第 2 节)
11 vector<int> parent;
12 int find(int x) { ... }
13 void unite(int x, int y) { ... }
14
15 int kruskal(int n, vector<Edge>& edges) {
16     parent.resize(n + 1);
17     for (int i = 1; i <= n; i++) parent[i] = i;
18
19     sort(edges.begin(), edges.end());
20     int mstWeight = 0, edgeCount = 0;
21
22     for (Edge& e : edges) {
23         if (find(e.u) != find(e.v)) {
24             unite(e.u, e.v);
25             mstWeight += e.w;
26             edgeCount++;
27             if (edgeCount == n - 1) break;
28         }
29     }
30     return edgeCount == n - 1 ? mstWeight : -1; // -1 表示不连通
31 }
```

6.2 Prim 算法 (加点法, $O(n^2)$)

Listing 21: Prim 朴素版

```

1  const int INF = 0x3f3f3f3f;
2  int graph[505][505];
3
4  int prim(int n) {
5      vector<int> dist(n + 1, INF);
6      vector<bool> visited(n + 1, false);
7      dist[1] = 0;
8      int mstWeight = 0;
9
10     for (int i = 1; i <= n; i++) {
11         int u = -1, minDist = INF;
12         for (int j = 1; j <= n; j++)
13             if (!visited[j] && dist[j] < minDist) {
14                 minDist = dist[j];
15                 u = j;
16             }
17         if (u == -1) return -1; // 不连通
18         visited[u] = true;
19         mstWeight += dist[u];
20
21         for (int v = 1; v <= n; v++)
22             if (!visited[v] && graph[u][v] < dist[v])
23                 dist[v] = graph[u][v];
24     }
25     return mstWeight;
26 }

```

7 关键路径 (AOE 网)

Listing 22: 拓扑排序求关键路径

```
1 struct Edge { int to, w; };
2
3 // 正推求 ve (最早发生时间)
4 vector<int> topoOrder(int n, vector<vector<Edge>>& adj,
5                       vector<int>& indeg, vector<int>& ve) {
6     queue<int> q;
7     for (int i = 1; i <= n; i++)
8         if (indeg[i] == 0) q.push(i);
9
10    vector<int> topo;
11    while (!q.empty()) {
12        int u = q.front(); q.pop();
13        topo.push_back(u);
14        for (Edge& e : adj[u]) {
15            indeg[e.to]--;
16            if (indeg[e.to] == 0) q.push(e.to);
17            ve[e.to] = max(ve[e.to], ve[u] + e.w);
18        }
19    }
20    return topo;
21 }
22
23 // 逆推求 vl (最晚发生时间)
24 void calcVl(int n, vector<int>& topo, vector<vector<Edge>>& adj,
25            int totalTime, vector<int>& vl) {
26     fill(vl.begin(), vl.begin() + n + 1, totalTime);
27     for (int i = n - 1; i >= 0; i--) {
28         int u = topo[i];
29         for (Edge& e : adj[u])
30             vl[u] = min(vl[u], vl[e.to] - e.w);
31     }
32 }
33
34 // 关键活动: ve[u] == vl[u] 且 ve[u] + w == vl[v]
```

8 常见技巧与模板

8.1 快速 I/O

Listing 23: C++ 加速 I/O

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr);
3 // 注意：不可混用 scanf/printf 和 cin/cout
```

8.2 INF 常量

```
1 const int INF1 = 0x3f3f3f3f;    // 1.06e9, 两个相加不溢出
2 const int INF2 = INT_MAX / 2;    // 更保守
```

8.3 重边处理

```
1 if (w < graph[u][v]) {          // 保留最小权值
2     graph[u][v] = w;
3     graph[v][u] = w;            // 无向图
4 }
```

8.4 邻接表排序

```
1 for (int i = 0; i < n; i++)
2     sort(adj[i].begin(), adj[i].end());
```

8.5 DFS 检测有向环

Listing 24: 三色标记法

```
1 bool hasCycle(int u, vector<vector<int>>& adj, vector<int>& color) {
2     color[u] = 1;    // 正在访问
3     for (int v : adj[u]) {
4         if (color[v] == 1) return true;    // 后向边 → 有环
5         if (color[v] == 0 && hasCycle(v, adj, color)) return true;
6     }
7     color[u] = 2;    // 已访问完
8     return false;
9 }
```

```

10
11 // 调用:
12 // vector<int> color(n, 0);
13 // for (int i = 0; i < n; i++)
14 //     if (color[i] == 0 && hasCycle(i, adj, color)) ...

```

8.6 索引排序

Listing 25: 按值排序返回原下标

```

1 vector<int> isort(const vector<int>& arr, int n) {
2     vector<int> ia(n);
3     for (int i = 0; i < n; i++) ia[i] = i;
4     sort(ia.begin(), ia.end(), [&](int a, int b) {
5         return arr[a] < arr[b];
6     });
7     return ia;
8 }
9 // ia[0] = arr 中最小值的下标

```

8.7 优先队列（堆）

Listing 26: priority_queue 用法

```

1 // 最大堆（默认）
2 priority_queue<int> pq;
3
4 // 最小堆
5 priority_queue<int, vector<int>, greater<int>> pq_min;
6
7 // 自定义类型（小根堆）
8 struct Node {
9     int dist, id;
10    bool operator>(const Node& o) const {
11        return dist > o.dist;
12    }
13 };
14 priority_queue<Node, vector<Node>, greater<Node>> pq;

```

附录：算法速查表

算法	时间复杂度	核心数据结构
并查集 Find	$O(\alpha(n))$	parent[] 数组
并查集 Unite	$O(\alpha(n))$	rank[] 数组
DFS	$O(n + m)$	递归 / stack
BFS	$O(n + m)$	queue
拓扑排序	$O(n + m)$	indeg[] + queue/stack
Dijkstra 朴素	$O(n^2)$	dist[] + visited[]
Dijkstra 堆优化	$O(m \log n)$	priority_queue
Floyd	$O(n^3)$	dist[][]
Bellman-Ford	$O(nm)$	Edge[]
Kruskal	$O(m \log m)$	Union-Find
Prim 朴素	$O(n^2)$	dist[] + visited[]

——祝考试顺利！——